We configured the `Retry` filter to be triggered when an `IOException` or `TimeoutException` occurs. In this case, since the downstream service is not running, the exception thrown is of type `ConnectException`, so the request is conveniently not retried, which allowed me to show you the combined behavior of circuit breakers and fallbacks without retries.

So far we have looked at patterns that make the interactions between Edge Service and the downstream applications more resilient. What about the entry point of the system? The next section will introduce rate limiters, which will control the request flow coming into the system through the Edge Service application. Before proceeding, stop the application's execution with Ctrl-C.

9.3 Request rate limiting with Spring Cloud Gateway and Redis

Rate limiting is a pattern used to control the rate of traffic sent to or received from an application, helping to make your system more resilient and robust. In the context of HTTP interactions, you can apply this pattern to control outgoing or incoming network traffic using client-side and server-side rate limiters, respectively.

Client-side rate limiters are for constraining the number of requests sent to a downstream service in a given period. It's a useful pattern to adopt when third-party organizations like cloud providers manage and offer the downstream service. You'll want to avoid incurring extra costs for having sent more requests than are allowed by your subscription. In the case of pay-per-use services, this helps prevent unexpected expenses.